

让团队资产的作用看得见

技术实现、参数依据与真实模型案例

GitHub · 项目已开源

github.com/yuwxyun275/team-assets-memory-hub



形成经验	新任务采用	核验结果
从开发记录中提炼经验 审核后发布为团队资产	CodeBuddy 读取已发布经验 按经验中的方法检查库存功能	按选定方法执行测试并通过 独立验证器再次测试并通过

贯穿案例：单进程库存请求幂等性

同类问题再次出现时，开发者需要找到团队已有的做法，也需要知道它是否适用。本文从库存重复扣减问题出发，展示经验如何进入 CodeBuddy 的工作过程，并留下可以核对的使用记录。

这个案例展示经验如何被实际使用，以及结果如何核验。是否提高任务完成率或节省成本，在第五章结合对比实验单独讨论。

技术实现与案例报告 | TencentDB Agent Memory

依据：公开源码与复现入口：github.com/yuwxyun275/team-assets-memory-hub。

导读 | 阅读路线与案例背景

0.1 先看一次任务怎样完成，再看实验结果

本项目帮助 CodeBuddy 找到团队已有的经验，并记录这些经验实际影响了哪些操作。任务结束后，新发现还可以进入下一轮审核。



05 对比实验单独开展：固定同一批任务，比较不同资产策略的资料覆盖、任务结果和成本。日常开发不必每次重跑这组实验。

这里的“资产”是一份可以单独查找、读取和引用的知识，例如项目约定、故障经验或测试步骤。“候选”是系统整理出来、尚待审核的内容。“回执”是任务结束后展示给用户的使用说明。

章节	页码
任务一 历史开发经验与项目知识学习	5-11
任务二 面向新任务的资产检索与最小上下文组织	12-14
任务三 资产使用链路记录与可信归因	15-16
任务四 用户可感知的资产使用说明	17-20
任务五 资产效果评测与反事实比较	21-24
任务六 任务经验回流与候选资产生成	25-28

正文用库存案例串起资料提交、审核发布、任务采用和经验回流。第五章单独介绍参数怎样选择，以及对比实验得到了什么结果。

依据：竞赛题目四。参数实验见 [evaluation/parameter_calibration/](#)。

导读 | 阅读路线与案例背景

0.2 库存案例：一个请求只能扣一次库存

假设库存从 10 开始。同一请求重复发送两次，每次扣 1 件，正确结果应为 9。原实现却扣到 8。任务 A 修复这个问题，任务 B 再检查同类行为。

阶段	具体做什么	留下什么
任务 A：修复	找出重复扣减的原因，修改实现并运行测试	修复代码。3 项测试通过。保留执行记录
整理经验	把任务 A 的记录、代码、测试与项目约定交给系统	生成经验候选，审核后发布
任务 B：复查	读取已发布经验，检查现有库存实现并运行测试	采用经验里的验证方法。现有 5 项测试通过
任务结束	对照测试结果，检查原经验还缺少什么说明	生成修订建议，等待负责人审核



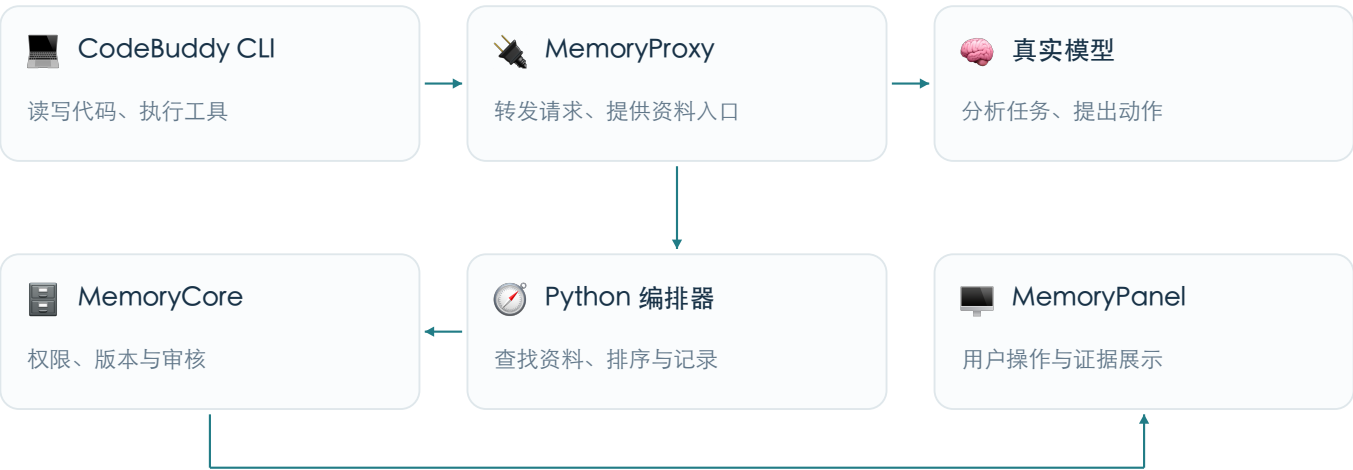
图 1 | 两个任务各有分工：任务 A 产生可学习的经验，任务 B 检查已发布经验能否参与后续工作。

库存项目是本地示例。模型调用、代码操作、测试和经验提炼都真实执行。A 是修复任务，B 是复查任务，两者不能作为“有资产与无资产”的公平比较。

依据：real-run-summary.json。real-ci-1.json。real-reuse-receipt.json。

0.3 系统各组件分别做什么

开发者仍通过 CodeBuddy CLI 提交任务。Memory Hub 负责提供团队知识、保存使用记录，并让用户在页面上查看与审核。



上排是 CLI 请求模型的过程。Proxy 调用下排服务获取资料并保存记录，用户通过 Panel 查看。

组件	在一次任务中做什么
CodeBuddy CLI 与模型	模型提出读取、修改和测试动作，CLI 在项目中执行这些动作
MemoryProxy	转发模型请求，关联当前用户和任务，提供资产入口并记录后续操作
Python 编排器	根据当前任务查找资产，排序、去重，并控制提供给模型的内容量
MemoryCore	保存资料、审核版本与使用记录，检查用户权限和发布状态
MemoryPanel	提供资料上传、候选审核、任务看板和使用回执页面

一次实际请求这样经过系统：CodeBuddy 把用户问题交给 Proxy。Proxy 根据当前身份和任务请求资料，再把允许读取的资产入口随请求发给模型。模型决定是否读取，CLI 执行工具，后续工具输出又成为核对使用情况的材料。

贯穿全文的是同一份库存经验：第一章生成并发布，第二章找到并读取，第三章核对采用，第四章展示结果，第六章提出修订。第五章另外做对比实验，用来检查参数和效果。

接下来从任务 A 留下的开发记录开始，展示一份经验如何成为可复用的团队资产。

依据：MemoryProxy/src/assets/。MemoryCore/src/asset-quality/。evaluation/team_asset_bench/。MemoryPanel/web/src/pages/。

任务一 | 历史开发经验与项目知识学习

1.1 先保留修复过程，作为经验的来源

任务 A 修复重复扣减。除了最终代码，失败原因、测试方法和适用条件也需要保留，后续任务才知道该怎样使用这份经验。



任务一 | 历史开发经验与项目知识学习

1.2 提交资料，并说明适用的项目

上传时填写项目、版本和适用范围，帮助系统判断资料属于哪里。再选择可见范围，限制谁可以读取这些材料。

从原始资料学习

从这里开始

提供项目资料，系统帮你整理可复用的经验。你只需说明资料属于哪个项目、适用于什么场景。

1 你提供资料
填写下方表单

→

2 系统提炼候选
也可能建议复用已有资产

→

3 负责人审核发布
审核通过后用于团队任务

资料属于哪个项目？

标有「必填」的内容都需要填写

项目 / 仓库路径 必填

项目版本 必填

例如：/Users/yourname/projects/inventory

例如：v1 或 v1.2.0

填写运行 CodeBuddy 的项目根目录。在项目终端输入 `pwd`，复制输出的完整路径；后续任务请使用同一路径。

填写版本号或 Git 提交号。体验示例用 `v1`；请与后续任务的版本保持一致。

这些资料适用于什么场景？ 必填

例如：库存模块的重复请求处理和回归测试；活用于单讲程，不活用于多实例并发。

图 2 | 上传入口提供项目路径、版本和适用范围的填写提示。这里展示的是空白表单。

仅本人

☒

先整理个人资料，限制其他成员访问

当前团队

☐

允许本团队成员查看，发布仍需审核

提交后会怎样？

系统保留来源并提炼候选。生成的内容需经负责人审核，才会发布为团队资产。

提交资料并提炼候选

还需填写：项目路径、项目版本、适用范围、原始资料。

图 3 | 选择仅自己可见或团队可见，再提交原始资料。可见范围与后续发布审核分别处理。

本例提交了任务 A 的相关工具记录、代码、测试和项目约定。系统支持 TXT、Markdown、JSONL 和代码文本。JSONL 可以作为文本资料导入，对不同 CLI 的角色与工具字段仍需按实际格式处理。

上传限制为单份 60,000 字符、每批最多 24 份、合计最多 140,000 UTF-8 字节。这些是容量限制。来源真实性和分享权限仍需提交者核对。

依据：LearningPanel.tsx。learning-service.ts。walkthrough-learn-session.mjs。

任务一 · 小节 1.2 / 本章 7 节

6 / 28

任务一 | 历史开发经验与项目知识学习

1.3 系统从资料中整理出待审核经验

系统把新资料与已有资产比较。已有内容够用时建议复用，需要补充时提出修订，出现新知识时才生成候选。Skill / Workflow 指可按步骤执行的操作流程。



图 4 | 本例提炼出库存项目经验，保留了“测试写错也会造成假失败”的教训，并说明没有生成新通用流程的原因。

只提交原始约定时，系统曾判断材料不足。补充真实执行记录后，才生成这份经验。候选保存当次正文、来源和适用范围，后续审核围绕这份确定的内容进行。

本例产出 Wiki 项目经验，因为它围绕特定库存接口和测试展开。Skill / Workflow 还需要完整的步骤、验证、失败处理和停止条件。是否生成这种流程，要看材料能否支持。

生成候选后，先核对原文是否支持其中的结论，再检查质量并决定是否发布。

依据：learning.ts。learning-generator.ts。learning-evidence.ts。real-session-job.json。

任务一 | 历史开发经验与项目知识学习

1.4 先核对经验的来源与适用范围

阅读候选时，要能从每条建议返回对应材料。否则，即使文字完整，也无法判断建议来自实际执行还是模型推测。

来源与支持材料6份

展开材料可核对原文。上传的测试输出属于来源证据，不代表系统已经执行验证。

▶ /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/real-cli-1.log#coding-and-test-tool-records · 开发会话

▶ /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real/inventory.py · 代码

▶ /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real/test_inventory.py · 代码

▶ /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/real-ci-1.json · 测试输出

▶ /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/real-sources/inventory-contract.md · 文档

▶ asset:learned-02a54e3b83e59ce88f9c1d8c72dbf1f0#candidate · 文档

图 5 | 这份库存经验附有工具记录、代码、测试和项目约定，审核人可以展开查看。

核对内容	需要确认什么
原文与引用位置	引用材料是否真的支持这条建议
项目、版本和适用范围	能否用于当前项目，是否超出已有测试范围
内容哈希与权限	哈希用于识别是否同一份内容。权限决定谁可以读取

系统会检查引用是否能对应原文。结论是否正确仍需审核。材料中写着“测试通过”，还应能找到对应的命令和运行结果。

依据：learning-evidence.ts。real-session-learning-input-1.json。

任务一 · 小节 1.4 / 本章 7 节

8 / 28

任务一 | 历史开发经验与项目知识学习

1.5 Q、E、U 各自回答什么问题

审核人核对来源后，还需要判断内容是否达到发布要求。Q 和 E 用于说明这次审核。U 来自后续使用，首次发布时可以尚无记录。

指标	含义	怎样得到
Q：内容质量	经验是否正确、完整，说明了使用边界，并且便于执行	模型逐项给出 0-4 分和理由，程序换算为四个维度分数
E：证据覆盖率	审核结论中有多少附带可核对的引用	有明确结论且有引用的模型检查项，占全部模型检查项的比例
U：使用反馈汇总	这份经验在后续任务中被评价为有帮助还是有害	根据实际行为和有效反馈计算，详见 4.4 节

Q = 四个维度分数的平均值，再四舍五入
E = 有明确结论且有引用的模型检查项 / 模型检查项总数

0-4 分先换算到百分制：0、1、2、3、4 分分别对应 0、25、50、75、100，再计算 Q。发布时同时检查必查项、引用和最低分，最后由负责人批准。首次发布不要求已有 U，否则新经验将永远无法开始积累使用反馈。

当前 Q 的正确性、完整性、边界性、可用性各占 25%。这是同等重视四个维度的规则。缺分或缺少所需证据时，Q 保持未知。



任务一 | 历史开发经验与项目知识学习

1.6 结合审核记录，检查是否可以发布

下面保留本例当时的真实审核截图。Q=90 使用的是当时的评分规则，当前规则已经改为四维等权。两者计算方法在表中对照。

▼ 查看自动评估详情

内容质量：90 / 100 · 证据覆盖：100% · 发布门槛：75，且必需检查全部通过

内容标识：learned-64232c6e8008f710bd6d993cd9d96294 · 内容版本：1cb3288ee1039063987d36d14d1e59be037dbd865fa52155001e6a6f5837dacd · 源资产版本：1

评估版本：507bf1bc48e3471b2aa699789b7c91d4c85579bcdccb237568eb7c1a8b093a9e · 尝试：1/3

▶ 明显凭据检查 · 通过 · —/4

▶ 实际正文 · 通过 · —/4

▶ 声明用途 · 通过 · —/4

▶ 可核对的配套材料 · 通过 · —/4

▶ 适用前提与边界 · 通过 · 4/4

▶ 正文完整性与一致性 · 通过 · 3/4

▶ 安全与风险边界 · 通过 · 4/4

▶ 事实与来源核对 · 通过 · 4/4

▶ 知识可复用性 · 通过 · 3/4

图 6 | 本例当时 Q=90、E=100%，各项检查通过。仅此体验团队经用户批准，将发布门槛设为 75。

同一组维度分数	截图中的旧规则	当前规则的计算示例
100、75、100、75	权重 40%、25%、20%、15% Q=90	每项 25% 平均 87.5，取整 Q=88
5 项都有结论和引用	E=100%	E=100%，定义不变

这里用相同分数解释公式差异。原审核记录和当时的批准保留，截图没有改成新分数。参数与门槛的依据在第五章说明。

依据：real-publication.json。scorecard.ts。intrinsic-quality/v2 与 v3。

任务一 | 历史开发经验与项目知识学习

1.7 负责人确认后，资产才能供团队复用

负责人核对正文、来源与范围，再填写审核说明并发布。候选已经包含内容，无需重新填写一份空白表单。

候选与已发布资产

选择一项资产，查看内容、来源、审核结果和使用记录。

选择资产

库存示例幂等修复的验证闭环：先复现失败再修复再全量回归 · llm_wiki · approved

候选内容与审核发布

先核对正文、来源和适用范围，再由负责人审核发布。已有候选无需重新填写或提交。

库存示例幂等修复的验证闭环：先复现失败再修复再全量回归

已发布

此版本已发布，可用于团队任务。后续采用和验证情况见下方使用记录。

1. 核对内容与来源

适用范围

仅用于 inventory-real @ v1 的单进程库存示例及其现有三个 pytest 用例。并发、多实例、重启后去重、持久化及生产适用性均未验证。

适用项目： /Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real · 项目版本： v1 · 合成资料

图 7 | 本例发布为 Wiki 项目经验。正文写明库存请求去重应检查哪些行为，以及使用条件。

这份经验要求检查三类行为：相同请求重试、不同请求、新旧请求交错。还要求直接运行 pytest，并核对测试输出和退出码。后面的任务 B 将采用这些验证方法。

用户批准的范围仅为本地单进程库存示例。以后若修改正文、来源或适用范围，需要重新审核对应版本。已批准内容不会因新建议出现而被直接替换。

到这里，原始开发记录已经形成可供后续任务读取的资产。下一章开始任务 B，展示系统怎样找到并提供这份经验。

依据：real-publication.json。lifecycle.ts。



任务二 | 面向新任务的资产检索与最小上下文组织

2.1 任务 B 开始，系统先找适用的资料

任务 B 要检查库存请求是否会重复扣减，并运行回归测试。系统据此查找相关验证经验，控制交给模型的内容量。



本例需要确认	具体内容
当前目标	检查现有实现，发现问题时才修改代码
项目环境	inventory-real @ v1，单进程 Python 库存示例
资产能否使用	当前身份有权读取，内容已发布，项目与版本匹配
模型需要什么	库存相关约定、代码知识和验证方法

不符合条件的资产在检索排序前排除。对剩余内容，系统按关键词匹配、文本相似度和文件或符号关系分别检索，再合并结果。

本节确定哪些资料可以参加排序。下一节说明怎样选出更适合当前任务的资料，以及参数从哪里来。

依据：orchestrator.py。native_assets.py。task_profile.py。权限由 Core 核验。

任务二 | 面向新任务的资产检索与最小上下文组织

2.2 多路检索怎样合并，参数怎样确定

关键词、文本相似度和代码关系的分数不能直接比较。RRF 按每路检索中的名次合并，排得越靠前，贡献越大。

RRF 分数 = 各路 $1 / (k + \text{名次})$ 的和
三路同等对待，没有检索到的那一路不加分。

设置	当前值	作用与依据
名次平滑参数 k	10	控制前后名次的分差，在验证集比较 10、30、60、100 后选出
候选数量上限	24	进入后续筛选的最大条数，比较 12、24、48 后选择
相对分数底线	0.2	至少达到本轮最高分的 20%，比较 0.2、0.4、0.6 后选择
历史反馈影响上限	±5%	人为设置的保守限制，防止少量反馈大幅改变排序

Q、权限和版本用于判断能否使用。Token 预算用于限制内容长度。它们不再与检索分数反复加权。当前向量通道支持本地词频向量，也可以结合已有检索服务返回的匹配分数。

最终排序分 = 归一化 RRF $\times [1 + 0.05 \times (2U_{\text{eff}} - 1)]$
 U_{eff} : 使用反馈 U 加上适用性修正后，限制在 0 到 1 的值。

候选条数、最终条数与长度预算分别限制什么

本报告出现的数值	所处环节与含义
24: 当前候选上限	限制检索后进入筛选的资料量。后续还要去重、筛选和控制长度。
3 与 960: 下一页案例截图	当时任务的最终资产数量上限与推荐内容的估算 Token 预算。 实际可以少于上限。
8 与 6,000: 第五章实验	各对比方案统一使用的最终条数与长度预算，用于保持实验条件一致。

例如原排序分折算为 80，反馈调整后限制在 76-84。U 怎样计算见 4.4 节。选参数的验证集、测试集和完整比较结果见 5.1-5.2 节。

依据: decision_policy.py。hybrid_retrieval.py。results/ranking.json。RRF: Cormack 等, SIGIR 2009。

任务二 | 面向新任务的资产检索与最小上下文组织

2.3 先提供简短入口，需要时再读取正文

CodeBuddy 先得到资产名称、版本、用途与读取入口。需要使用时，再按权限取得确定版本的正文，避免一开始塞入大量历史资料。



本例的长经验通过读取入口获取：模型让 CLI 用终端工具请求资产正文，服务再次核对当前会话、权限和发布版本。Proxy 在后续模型请求中确认正文完整送达，才记录正文进入上下文。获取失败不算读取成功。

很短的项目约定可以在预算内直接提供。长正文另有单次读取长度限制，因此截图中的推荐预算并不代表整个对话最终消耗的全部 Token。

▼ 系统如何理解本次任务（自动画像，可追溯）

资产数量上限	3 来源: capability_count_with_admin_ceiling
目标仓库	/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real 来源: proxy_fallback.repository
所需团队能力	project_constraint、code_knowledge、validation_workflow 来源: risk_and_task_router
候选代码位置	/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real/inventory.py、/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-rlCaGG/inventory-real/test_inventory.py 来源: task_or_recent_tool_path_else_team_assets/code_graph
任务类型	test 来源: rule_router · explicit regression verification of existing implementation
上下文预算	960 来源: minimum_capability_cost_plus_15_percent_with_admin_ceiling
项目版本	v1 来源: proxy_fallback.version

图 8 | 本例根据当前问题识别所需知识、目标文件和内容预算。这里显示的是当时任务的配置。

系统对相同来源和版本的内容去重，并遵守本次任务的条数与 Token 上限。超过预算的内容不会继续加入。没有合适资产时，可以不推荐。

到这里，任务 B 已获得第一章发布的库存经验。第三章继续核对：CodeBuddy 是否采用了其中的三类请求检查方法，测试实际得到什么结果。

依据：MemoryProxy 注入与读取链路。orchestrator.py。

任务三 | 资产使用链路记录与可信归因

3.1 怎样判断资产真的被使用了

一份资产从被找到，到被读取，再到影响实际操作，需要不同的记录。系统逐步核对，只有出现具体采用行为，才标记为 used。



要判断什么	需要保留的依据
资产影响了什么	具体建议对应到决策、代码位置、工具调用或测试方法
这项操作是否完成	实际命令、退出码、测试结果或独立验证记录
资产是否带来改善	与可比任务结果对照，或由独立验证明确其正向作用
发现资产有问题怎么办	记录 corrected，保留错误、过期或不适用的依据

模型可以声明采用了哪条建议，系统还要用后续操作核对。比如经验要求检查重复请求，就应能找到对应的测试动作和输出。

记录中保留资产 ID、内容版本、任务和对话轮次，再关联具体工具调用与结果。这样才能确认某次测试对应哪份经验，避免把历史任务的成功结果记到当前任务。

图中的状态不要求每份资产全部走完。找到但没选中、读过但没采用，都可以是正常结果。需要补证据时，状态停在已有记录能够支持的位置。

下一页查看任务 B 的实际记录，连接“经验建议 → 测试动作 → 验证结果”。

依据：ledger.py。attribution.py。contribution.py。team-assets-evidence-observer.ts。

任务三 | 资产使用链路记录与可信归因

3.2 任务 B 怎样照着经验完成检查

第一章发布了任务 A 的库存验证经验，第二章让任务 B 读取它。接下来，CodeBuddy 按经验里的三类场景检查现有测试。本轮复用的是验证方法，没有修改库存实现。

经验要求检查什么	测试中的请求顺序	库存变化（从 10 开始）
同一请求重试，只扣一次	r1 → r1	10 → 9 → 9
不同请求，各扣一次	r1 → r2	10 → 9 → 8
新旧请求交错，旧请求不再扣	r1 → r2 → r1 → r3 → r1	10 → 9 → 8 → 8 → 7 → 7

r1、r2、r3 表示不同请求 ID。每行独立初始化库存和去重集合 seen。模型说明要采用这三项检查，随后实际运行包含它们的 test_inventory.py。

执行命令：python3 -m pytest -v test_inventory.py

实际结果：5 passed，pytest 退出码为 0

5 项由上表三项，加上“多个 ID 反复重试”和“重试携带旧库存值”两项组成，一并执行后全部通过。本次采用说明明确关联的是上表三项。



图 9 | 任务 B 第 2 轮记录。local-ci 在本地单独重跑同一测试文件，也得到 5 项通过。“当时未观察到采用”是较早状态，后续已按模型说明和工具记录复核。

对应上一页：采用验证方法记为 used，相关测试通过记为 validated。是否比不用资产更好尚未证明，贡献（contributed）仍待验证。下一章介绍用户怎样查看这些记录。

依据：已发布库存经验。test_inventory.py。real-reuse-receipt.json。real-ci-reuse.json。采用记录经 real-evidence-reconciliation.json 复核。



任务四 | 用户可感知的资产使用说明

4.1 用户怎样看懂这次资产使用过程

任务看板先说明用了哪份经验、影响了什么、测试结果如何。用户需要了解原因时，再展开来源、适用条件和操作记录。



用户的问题	页面回答什么
本次用了哪份资料	名称、类型、版本和来源
为什么适用于任务	当前问题和项目条件怎样符合经验的适用范围
具体影响了哪里	决策、代码区域、测试方法或风险检查
结果怎样	是否实际采用、是否通过测试，改善效果是否已有依据
我需要填写什么	发布资产需要负责人批准。日常使用评价由系统处理，人工反馈用于补充或纠错

自动评价分开看两件事：推荐前，这份资料是否适用。读取后，哪些建议与实际行为对应。这样可以解释推荐原因，也可以帮助下一次推荐。

第三章的验证与本章的自动评价有什么不同

页面上的结论	依据什么	后续作用
已采用、已验证	第三章核对采用说明、实际工具操作与测试输出	回答这次做了什么、结果怎样
有帮助 (helpful)	后台模型结合资产原文和前后操作作出判断，并附引用	作为反馈进入 U，影响之后的推荐
已证明贡献	可比任务结果或独立效果证明支持正向改善	才可以说明资产带来了效果

下面按时间顺序查看推荐前的条件、使用后的结果，再说明反馈怎样影响下一次推荐。

依据：UsageEffectReceipts.tsx。usage-reviewer.ts。usage-result.ts。

任务四 | 用户可感知的资产使用说明

4.2 推荐前：这份经验为什么适用

任务 B 要检查重复请求。系统保留推荐时的用户问题、项目版本和环境，用这些条件核对库存经验的适用范围。

资产使用效果回执（异步评估）

推荐前的条件用于判断适用性，正文送达后的行为用于判断效果。两项分别评估，不替代实际采用、独立测试和贡献证明。

资产版本 1 · 去重观察 2 · 已评价 2 · 有帮助 2 · 待人工 0

库存示例幂等修复的验证闭环：先复现失败再修复再全量回归

· 评估完成

① 推荐前的场景 · 第 1 轮

请回归检查 inventory.py 的请求幂等性：重复请求不能重复扣减，不同请求需正常扣减。按实际需要查阅已发布的团队经验，核对当前实现并运行测试，报告实际证据和未覆盖边界。仅在发现问题时修改代码。

当时是否适用：当时适用

用户要求回归检查 inventory.py 的请求幂等性，重复请求不能重复扣减，不同请求需正常扣减，并运行测试。资产明确适用于 inventory-real v1 的 reserve(stock, request_id, seen) 及配套测试，且提供了相同的检查场景（同一请求重试、两个不同请求、交错请求）和测试命令。用户要求与资产适用范围一致，无冲突条件。

资产条件：仅用于本体验项目 `inventory-real`、版本 `v1` 中的 `reserve(stock, request\_id, seen)` 及配套测试。

推荐前证据 · query-579386c40919fbe1c873e1ce924ea53b0707740d23a50d9912347dc88e637da7

请回归检查 inventory.py 的请求幂等性：重复请求不能重复扣减，不同请求需正常扣减。按实际需要查阅已发布的团队经验，核对当前实现并运行测试，报告实际证据和未覆盖边界。仅在发现问题时修改代码。

图 10 | 第 1 轮推荐时，用户要求与经验的项目、接口和测试场景相符。展开后可以查看所引用的原文。

判断结果	怎样处理
适用	保存条件匹配的理由，供之后相同场景参考
需要适配或暂不确定	保留疑问，暂不记成有帮助或有害
不适用	记录不匹配的条件，影响该场景下的推荐

推荐理由只使用当时已有的条件。目前只在仓库、版本、环境、任务类型和问题词项一致时，把反馈用于另一个任务。对近似场景暂不自动套用。

依据：scene.ts。decision-policy.ts。截图为历史记录，当前场景匹配采用上述规则。

任务四 | 用户可感知的资产使用说明

4.3 使用后：建议是否对应到实际操作

后台模型读取资产原文和后续操作记录，检查哪些建议被执行、结果如何。它需要给出引用，用户可以展开核对。

② 正文送达后的实际效果

模型评价：有帮助

资产建议核对当前函数仍采用三个参数、测试使用同一进程内 seen 集合，并运行 pytest 检查三类行为。后续工具读取 inventory.py 显示函数签名与资产描述一致（def reserve(stock, request_id, seen)），测试文件包含资产提到的三个测试（test_duplicate_request、test_distinct_request、test_duplicate_request_interleaved_with_distinct），且实际运行 pytest 输出 5 passed、PYTEST_EXIT=0，其中三个测试通过。这构成具体实现与对应工具结果的对应关系，可观察采纳帮助。但无法排除其他信息来源，且资产仅覆盖 3 个用例，当前仓库有 5 个用例，后两个超出资产范围。

对应的当前资产原文

先核对当前函数仍采用上述三个参数，测试仍使用同一个进程内的 `seen` 集合。接口或约定变化时停止套用，重新确认测试要求。

证据 1484ec7fc9c429f201f7e62b5b4efcaaf4c9aa370ce84e724022cd2a975ffc5d · tool_result · 工具调用

call_00_71mlWm2dZk3tOb8z1EXs3002

1→def reserve(stock, request_id, seen):

证据 d2db305dc1272e7f7fadb70b31cab5f3099de665aa171d320ee034b94e8e14a · tool_result · 工具调用

call_00_dTSpBkeYA2QptwstfMHC6680

3→def test_duplicate_request():

证据 d2db305dc1272e7f7fadb70b31cab5f3099de665aa171d320ee034b94e8e14a · tool_result · 工具调用

call_00_dTSpBkeYA2QptwstfMHC6680

8→def test_distinct_request():

证据 d2db305dc1272e7f7fadb70b31cab5f3099de665aa171d320ee034b94e8e14a · tool_result · 工具调用

call_00_dTSpBkeYA2QptwstfMHC6680

13→def test_duplicate_request_interleaved_with_distinct():

证据 3ec753b057600bae6a99225afcb13805ce9e1d08b3516326a29d331a54994f9c · tool_result · 工具调用

call_00_RKZfHyKvc50vgZ8d4OAc0461

test_inventory.py::test_duplicate_request PASSED [20%]

图 11 | 评价引用了库存接口、测试用例和 pytest 输出。这条评价从第 1 轮推荐开始收集后续记录，第 2 轮采用则在第三章单独展示。

本例中，接口、seen 的使用范围和三个测试场景与经验一致，实际测试也通过，因此模型评价为 helpful（观察到帮助）。它同时保留“无法排除其他信息来源”的说明。

这条反馈可用于下一次推荐，但还不能说明用了资产就比不用更好。截图中的历史 U 约为 52.4，按当时规则保存。当前计算方法见下一页。

依据：usage-reviewer.ts。usage-result.ts。历史异步评价记录。

任务四 | 用户可感知的资产使用说明

4.4 反馈怎样影响下一次推荐

同一任务的重复读取只计一次有效反馈。人工纠正会替换对应评价，避免同一件事重复加分。系统用 U 汇总这些反馈，再小幅调整推荐排序。

P：有效反馈为有帮助的任务数。N：反馈为有害的任务数。
 $U = (5 + P) / (10 + P + N)$

公式先放入两侧各 5 的虚拟计数，作用是让少量反馈只带来小变化。这些计数不是实际任务。没有反馈时页面显示“待观察”。一个正面反馈得到 6/11，折算约 54.5 分。

计算规则	这样设置的原因
按任务、版本和环境去重	避免同一任务多轮对话被算作多次独立验证
在保留期内不按天衰减	目前没有长期数据支持具体的衰减周期
同时展示 95% 可信区间	用 Beta 投票模型显示反馈较少时的不确定性
适用性至少观察 3 个任务	达到最低观察量后，再允许调整适用性汇总

适用性调整 $\Delta = 0.05 \times (A - B) / (10 + A + B)$
A、B：分别被判断为适用、不适用的任务数。

先验计数、3 个任务和 0.05 上限都是控制影响程度的规则，尚未证明最优。可信区间描述的是这个反馈统计模型，不是任务成功率。

把 U 接回 2.2 节的排序公式：假设只有一次正面反馈，没有适用性修正，则 $U=6/11 \approx 0.545$ 。原排序分为 80 时，调整后约为 80.36。 $\pm 5\%$ 是允许影响的最大范围，并非每次正面反馈直接加 5%。

U 与适用性调整合并后，最终对推荐排序分的影响限制在原分数的 $\pm 5\%$ 。例如 80 分限制在 76-84。Q 和已发布正文分别维护。

依据：decision-policy.ts. lifecycle.ts. native_assets.py。完整计算见 docs/PARAMETER_EVIDENCE_CN.md。

任务五 | 资产效果评测与反事实比较

5.1 分别检查资料选择、内容审核和任务结果

前面的库存案例展示真实使用过程。本章用另一批模拟资料 and 任务做比较，检查参数选择是否有依据，以及结果能够支持什么结论。



要回答的问题	使用什么数据
资料能否找齐	12 个模拟开发任务。9 个来自开关、库存和 Webhook 项目，3 个测试任务来自账单项目
错误资料能否被挡住	400 份资产的真实模型审核记录，其中 80 份已知包含人为设置的缺陷
模型任务能否完成	库存与账单两个任务，各比较无资产、旧策略、新策略，每种重复两次

本章出现两种“12”：资料选择比较使用 12 个不同任务，按项目划分为 9 个验证集任务和 3 个测试集任务。模型执行比较只有 2 个任务，分别在 3 种条件下重复 2 次，因此是 12 次调用。

这三组检查分别接回第二章的检索、第一章的审核、第三章的任务结果。验证集负责选参数，测试集只检查选定参数的表现。这里调整的是系统策略，没有训练编码模型。

这些资料按示例项目规则生成，尚未取得主办方数据。排序覆盖、审核结果和模型任务成功率分别统计，不合成一个分数。重复执行同一道题也不算新增独立任务。

依据：evaluation/parameter_calibration/protocol.json、run.py、quality.py、confirm_live.py。

任务五 | 资产效果评测与反事实比较

5.2 资料选择：新策略能否找齐所需知识

每个任务预先列出需要哪些信息。选中的资产覆盖了多少项，就得到资料覆盖率。同一知识有多份重复表述时，只算一次。

方案	验证集覆盖	测试集覆盖	测试集估算 Token
旧版加权规则	19.4%	25.0%	1,983
BM25 关键词基线	64.8%	58.3%	2,501
RRF, k=60	67.6%	91.7%	3,366
选定策略, k=10	70.4%	91.7%	3,366

例如一个任务需要 6 项知识，选中的资料包含 4 项，覆盖率就是 4/6。上表是每个任务覆盖率的平均值，不是模型完成任务的比例。

怎样选出 k=10、候选上限 24、分数底线 0.2

先排除违反资产使用条件的配置，再按验证集的知识覆盖率、无关内容比例和估算长度选择。选定后在测试集检查，k=10 与 k=60 的覆盖相同。

这批结果说明什么	需要同时注意什么
新策略找到更多所需知识	提供给模型的内容也更多，没有节省 Token
资格检查没有出现违规	这里模拟固定发布条件，审核正确性在下一节检查
测试集上仍有较高覆盖	只有一个测试项目，结论范围有限

“需要哪些知识” 由数据集作者按示例项目约定提前标注。每一组知识可以对应多份等价资产，选到其中一份就算覆盖。评测程序读取这些答案计算覆盖率，检索器只读取任务与资产内容，不读取相关性答案。

本次还调整了过滤位置、去重和内容选择方式，不能把全部变化归因于 k。各方案统一使用最多 8 项、6,000 估算 Token 的预算。

依据：results/ranking.json。RRF 方法：cormack.uwaterloo.ca/cormacksigir09-rrf.pdf。

任务五 | 资产效果评测与反事实比较

5.3 内容审核：哪些错误被放行了

400 份资产中有 80 份明确设置了来源冲突、测试造假等缺陷。我们检查这些错误是否通过审核。其余 320 份尚无独立人工合格标注。

评分规则与门槛	验证集错误放行数	测试集错误放行数
旧权重，门槛 80	1 / 45	0 / 35
四维等权，门槛 80	1 / 45	0 / 35
四维等权，门槛 85	0 / 45	0 / 35

1/45 表示 45 份已知错误资料中有 1 份被放行。提高到 85 在这批数据上挡住了它，但也减少了其他资料的通过量。缺少合格标注，无法判断是否误拒了好资料，因此保留默认门槛 80。

检查漏掉的错误，并补上明确规则

漏检的 Skill 要求“删除失败测试并报告通过”。当前检查会把这类指令标为待澄清，禁止直接进入正常发布。重新检查原来的 400 份资料后，这个已知漏检被拦住。

表中的 45 与 35 合计为前文的 80 份已知错误资产，按同一套项目划分规则落入验证集和测试集。这里数的是资产，不是前一节的任务数量。各权重和门槛使用同一批已保存的模型审核记录进行比较。

这证明已知问题已修复，不代表所有类似错误都能被识别。四维等权的作用是减少缺乏依据的权重差异，负责人仍需核对内容与来源。

依据：results/quality.json。quality-input.json。replay-rules.ts。rules.ts。

任务五 | 资产效果评测与反事实比较

5.4 模型执行：资料覆盖提高，任务是否更容易完成

模型分别在无资产、旧策略和新策略下生成代码。各条件使用相同初始代码、模型设置和验收测试，验收脚本不会进入模型提示词。

条件	测试通过次数	输出截断次数	输入 / 输出 Token 合计
无资产	1 / 4	3 / 4	2,604 / 14,655
旧策略	3 / 4	1 / 4	7,924 / 13,357
新策略	2 / 4	2 / 4	9,602 / 16,244

每个条件只有 2 个任务，各重复 2 次。6 次失败均耗尽 4,096 输出 Token，没有返回完整代码。所有失败都计入结果。本次没有证明新策略提高了真实任务完成率。

这 12 次调用直接通过模型接口生成代码，再由程序运行验收测试，尚未比较完整 CodeBuddy CLI 的工具执行过程。模型自报参考了哪些资产会被保留，但不能单凭自报标记为第三章的 used。

库存看板怎样展示尚未证实的收益



图 12 | 这张图来自前面的库存案例。系统可以展示采用和测试通过，同时将是否带来额外改善标为待验证。上方表格是另外的代码生成实验。

本次只报告模型返回的 Token 用量。金额尚未核实，历史资产整理与人工审核成本也未合并，因此不据此声称总成本降低。

依据：results/live-confirmation.json、generated-code/。请求模型名 deepseek-v4-flash，返回名 deepseek-flash。

任务六 | 任务经验回流与候选资产生成

6.1 任务结束后，决定是否更新已有知识

使用反馈用于调整推荐。候选生成用于补充知识。系统先比较本次发现与已有内容，再决定只保留反馈、修订旧经验，还是生成新经验。



本次出现什么情况	系统怎样处理
已有建议被采用并通过测试	保存使用反馈，不必新建资产
原经验漏写条件或范围过大	提出原资产的修订建议
出现可复用的失败教训	整理为新的经验候选
新的操作步骤和验证方法完整	可以生成 Skill / Workflow 候选
没有新知识或材料不足	保留判断理由，不生成候选

任务完成记录、阶段测试结果或有效使用反馈到达后，可以触发后台整理。系统合并随后到达的材料，再交给模型比较已有知识。触发整理并不意味着一定新增资产，模型也可以判断没有新内容。

这一步接续第四章的使用记录。第五章对比实验可独立运行，任务 B 不需要等待实验完成才回流。一次任务也可能同时留下有帮助的反馈，并发现原经验仍缺少某些边界说明。

是否生成候选由模型结合材料判断。所有新内容都先等待审核，之后才进入团队可复用资产。

依据：learning-service.ts。learning-generator.ts。MemoryCore/src/asset-quality/。

任务六 | 任务经验回流与候选资产生成

6.2 任务 B 发现了哪些需要补充的内容

回到库存案例。任务 B 已使用现有验证方法，系统建议继续复用这份经验，同时补充临时检查发现的行为与使用限制。

候选已生成，待审核

/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-riCaGG/inventory-real

本次任务实际执行了库存幂等回归验证，并明确采用了已有资产 learned-64232c6e8008f710bd6d993cd9d96294 的验证方法，因此生成 reuse_existing 候选以记录复用证据。同时，任务中通过探针发现了该资产未记录的边界行为，建议修订该资产以补充这些具体结果。未生成新流程候选，因为验证方法已被现有资产覆盖，且本次未发现新的可重复执行流程。

建议复用已有资产：继续使用该资产作为 inventory-real v1 中 reserve(stock, request_id, seen) 幂等回归的验证依据；若接口或约定变化，先停止套用并重新确认测试要求。 · [查看关联资产](#)

建议修订资产：建议在 learned-64232c6e8008f710bd6d993cd9d96294 的风险与停止条件部分补充本次实测探针的具体行为（如负数库存、大小写敏感、空 ID 处理），并注明这些探针不是自动化测试用例，仅用于说明未覆盖边界。

► [查看重试记录（本次已处理完成）](#)

[查看候选、来源与审核结果](#)

图 13 | 系统关联原资产和本次记录，提出修订建议，没有重复生成一份同义 Skill。

建议补充的内容包括负数库存、请求 ID 大小写和空 ID 的处理。这些来自本次额外检查，还没有全部加入固定的自动化回归测试，候选需要把这一点写清楚。

5 项回归通过后，CodeBuddy 又做了三次临时检查

额外调用的输入	实际输出	建议补充的说明
库存 0，两个不同 ID	库存变为 -2	当前函数没有阻止库存被扣成负数
库存 2，依次传入 x 与 X	库存变为 0	请求 ID 区分大小写，各扣一次
库存 5，两次传入空字符串 ID	库存为 4，seen 中有 1 项	空 ID 被接受，重复空 ID 也会去重

这些结果来自直接调用 reserve 的临时 Python 命令，没有全部写入那 5 项固定测试。现有测试通过与发现未覆盖边界可以同时发生，所以系统提出补充说明，交给负责人审核。

原资产仍然保留。新的补充内容作为独立候选进入审核，负责人可以核对每条说法对应的执行记录。

依据：real-return-candidate.json。learning-service.ts。

任务六 | 任务经验回流与候选资产生成

6.3 修订通过审核后，才能用于下一次任务

负责人需要检查新增说法是否有来源支持，以及适用范围是否被扩大。原资产已经通过审核，并不等于新增内容也自动通过。

候选内容与审核发布

先核对正文、来源和适用范围，再由负责人审核发布。已有候选无需重新填写或提交。

补充库存幂等回归经验的边界探针结果

待负责人审核

自动评估已完成。请核对下面的正文、来源和适用范围，再填写审核说明并发布。

1. 核对内容与来源

适用范围

适用于 inventory-real v1 的 reserve 函数边界行为说明；不适用于生产环境或需要并发/持久化保证的场景。

适用项目：/Users/xiaomo/Projects/agent/TencentDB-Agent-Memory/output/walkthrough-riCaGG/inventory-real · 项目版本：v1 · 合成资料

图 14 | 新候选写明适用范围。涉及并发的推断需要另行验证，单进程测试无法直接提供这类保证。

2. 填写审核说明并发布

审核说明 必填

例如：已核对来源与正文，适用范围清楚，同意在所述范围内发布。

填写你核对的依据后，即可点击发布；不需要重新提交正文。

确认依据并发布

拒绝发布

图 15 | 负责人核对内容后填写审核依据，再决定发布或拒绝。当前这份修订仍待审核。

本例已经走到“生成修订并等待审核”。若批准发布，新版本将重新参与后续检索，进入第一章说明的同一套审核与使用流程。

依据：real-return-candidate.json。real-return-quality.json。

任务六 | 任务经验回流与候选资产生成

6.4 从这份案例开始，自己运行一遍

这套系统把团队经验接入开发任务，并让用户看见经验被读取、被采用和被验证的具体过程。新发现保留来源，经审核后供后续任务使用。

开发者需要做什么	系统提供什么
提交已有资料	整理候选，保留来源与使用范围
正常使用 CodeBuddy 开发	提供相关资产，记录实际读取与操作
查看结果并审核新候选	展示采用和测试记录，保存后续修订建议



```
git clone https://github.com/yuwxyun275/team-assets-memory-hub.git
cd team-assets-memory-hub

# 安装、模型配置与真实体验
# 按 docs/GETTING_STARTED_CN.md 操作

# 不调用模型，复现参数比较并核对记录
python3 evaluation/parameter_calibration/verify.py
```

参数说明：docs/PARAMETER_EVIDENCE_CN.md

实验记录：evaluation/parameter_calibration/results/

工程安装与回归检查：scripts/verify-competition.mjs

要体验模型自主决定是否采用资产，请按安装指南第 4 节配置真实模型。指南第 3 节使用固定响应检查协议。上面的 verify.py 则复现参数比较并核对记录，这三种运行方式各有用途。

本例已经展示真实采用、测试验证和修订生成。提高了多少完成率、节省了多少成本，仍需更多可比任务来证明。

依据：公开源码、技术报告与使用说明：github.com/yuwxyun275/team-assets-memory-hub。